Noughts and Crosses Design Document

Mohamed El Bakry

Initiated on December 9, 2024
February 13, 2025

Contents

List of Figures

List of Tables

1 Introduction

This document outline the design for a new project codenamed `UTTT-V3`. The former two versions being during the Intelligent Systems project assignment, and the minorly enhanced UI update and improve AI with more streamlined threading and some refactoring. The third version (V3) aims to build upon the previous to versions significantly by adding several properly-executed features which shall be covered in section 3

Naturally, the document shall delve into a technical overview and the project's architecture, including the server, the driving algorithms, and website structure. Additionally, the project will analyse the differing AIs and evaluate their performance. This will be done in section 6.

2 Background/Context

There is no noughts & crosses equivalent for lichess.org, an incredible, free, website rivalling that of paid and content-locked sites. As such, it is this projects goal to create a `li`-noughts&crosses∼*esque* website, that leads the ultimate tic tac toe / super noughts & crosses game.

Moreover, that fact indicates there's a strong gap which this project aims to fill.

3 Goals & Non-goals

Here we discuss the project's architecture, driving algorithms such as the AI ones, Elo, matchmaking, and multiplayer protocols.

As briefly mentioned in the introduction, the following are the goals of this project. Furthermore, to lead the Ultimate Tic Tac Toe space, the website must meet the following goals, and be executed to the highest quality possible. This ensures that website from its early days is seen as reputable, reliable, and trustwothy given the self-evident quality and standard. Here are some user experience (UX) enhancing goals.

3.1 Goals

- **Online/Local Multiplayer**: this encompasses the follwoing:
  - Open lobbies and spectating them.
  - Private, link-only invites.
  - **Correspondence mode** where a match is played over several days.
- **AI**: Medium, and Hard difficulty achieved by the current α-β pruning Minimax algorithm, and the Monte Carlo Treesearch (MCTS) algorithm, respectively.
- **Additional features**: computer analysis, optional game evaluation bar versus AI, match history, elo
- **Meta-level features**: Matchmaking, Match History

3.1.1 Future, long-term goals

- Tutorial
- Teaching tactics

3.2 Non-goals

- Social capabilities — instead a Discord server will be created to foster and grow the community

4 Technical Design

To ensure that effort expended on this project is efficient, it's important to align the project's objectives/goals, with its design and therefore the tools that shall fulfil that design. Thus, here we discuss each objective and its derivative design along with the technology that will achieve said goal (e.g. programming language, framework, library etc.). Docker

Table 1 showcases this.

Goal	Design	Tool
Multiplayer	Authoritative Server & Database	NoSQL
AI	Backend Minimax & MCTS	Golang/Rust Backend
Website features	Typescript & Web frameworks	Dart/Flutter

Table 1: Design overview.

Next, we break down each goal, the design decision and the respective tool.

4.1 Multiplayer: Authoritative Server & Database

To achieve multiplayer, there must be a way to synchronise multiple instances of games, and player actions with each other. Furthermore, an uncomprimisable tenet of multiplayer for player versus player games is competitive integerity. Thus, a server model which ensures said integrity is paramount to the project's success.

Therefore, an authoritative server model in which a central trusted node cross-checks and resolves game state differences, unintentionally or intentionally through cheating, is best to fulfil the promise and need for competitive integrity.

4.2 AI: Minimax & MCTS Server Backend

This section briefly explores the API's shape.

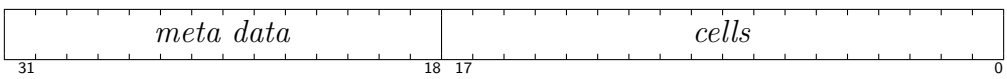
4.3 Architecture

Here we explore how the `ut3-oxide` Rust implementation will operate.

4.3.1 Binary Operations

For maximal efficiency's sake, we opt for representing each row as single `u32` integer, with 2 bits per cell ($9 \times 2 = 18$ — leaving 14 bits remaining for metadata)

Getting Bits



Formula

$$n\&((2^x - 1) \ll dx) \gg dx$$

Alternatively

$$n \,\&(((1 \ll x) - 1)) \ll dx \gg dx$$

NB: Naturally, if you don't need to offset the value, then dx is unrequired and is simplified to:

$$n \,\&(1 \ll x) - 1$$

Clearing Bits Here's how bits are cleared

$$n = 11011100$$
$$mask = 0b11 \ll dx$$
$$n = n \,\& \sim mask$$

$$n = n \wedge \neg(0b11 \ll dx)$$

n &= $\sim (0b11 \ll dx)$

Setting Bits

4.4 Components

4.5 Algorithms

As previously mentioned, there will be two AI Algorithms for the Player vs Computer mode. Furthermore, it's the Minimax Algorithm that ought ot be used for the easy difficulty. However, the relative difficulty depends on minimax's evaluation function, and the duration allowed for MCTS to explore the game tree. Therefore, it may be more efficient to develop the MCTS AI more thoroughly and create a perceived difficult through differing allowed tree-search time durations.

4.5.1 MCTS

4.6 Data Models

4.7 APIs and Interfaces

4.8 Tech Stack

Frontend: Flutter (Dart)
Backend: Rust SqlX?

5 Open Questions

6 Analysis & Evaluation

This section analyses the performance of the α-β pruning Minimax AI and the MCTS one. Thereafter, we compartively evaluate their respective performance.

What is more, the same will be done for the latency involving the server response times and the principle driving algorithms. Tables and figures are to assist.